Introduction :

Dans le cadre du module d'architecture des ordinateurs, nous devions réaliser un jeu de bataille navale programmé en assembleur RISC-V. Mon binôme ayant quitté la spécialité en il y a très peu de temps, j'ai réalisé l'intégralité de ce projet de manière individuelle.

Le sujet exigeait le développement d'un jeu opposant un joueur à l'ordinateur, où l'objectif est de torpiller une flotte placée aléatoirement par la machine. Enfin, le programme devait intégrer deux niveaux de difficulté : un mode "débutant", fournissant une indication lorsqu'un tir atterrit dans l'eau sur une case adjacente à un navire, et un mode "expert", sans aucune aide.

L'outil informatique utilisé pour créer ce projet est Rars que nous avions utilisé dans le cadre du TP en cours.

Analyse du travail à faire :

Dans un premier temps j'ai suivi les conseils donnés, c'est-à-dire, faire un étude sur papier du travail à effectuer. Comme pour une bataille navale à deux joueurs j'ai voulu implémenter le fait que l'ordinateur joue également. En effet le fait que le joueur ne puisse pas perdre ne me plaisait pas j'ai donc fait mon plan en intégrant le fait que l'ordinateur puisse également jouer.

Le plan que j'ai suivi est le suivant :

- Choix de la difficulté : débutant ou expert ;
- Initialisation des deux tableaux ;
- Remplissage du tableau joueur et du tableau ordinateur ;
- Boucle de jeu : Tant que le nombre de bateaux de l'ordinateur et du joueur est supérieur à 0 :
    - On demande au joueur la case sur laquelle il veut tirer ;
    - On vérifie s'il y a un bateau ou pas : s'il y a un bateau, on décrémente son nombre de bateau de 1, sinon on regarde s'il y a un bateau sur une case adjacente si on est dans le mode débutant ;
    - L'ordinateur choisit une case aléatoirement ;
    - On vérifie s'il y a un bateau ou pas : s'il y a un bateau, on décrémente son nombre de bateau de 1 ;
- Message de fin de partie
- Fin du programme

Pour avoir un programme clair il me fallait pour chaque situation un code pour le tableau :

- 0 s'il n'y que de l'eau
- 1 s'il y a un bateau
- 2 s'il y a eu un tir mais qu'il n'y avait que de l'eau
- 3 s'il y a un bateau qui a été détruit

De plus pour pouvoir réaliser ce programme j'ai identifié plusieurs fonctions qui seront appelé plusieurs fois.

Veuillez noter que dans la suite de ce rapport, nous appellerons « fonction » ces parties de programmes. Cependant celle-ci n'en sont pas exactement. En effet ces bribes de programmes ne peuvent être appelées que dans le programme main et non dans une autre fonction contrairement aux « vraies » fonctions vu dans le cours. Cette façon de faire est dû au fait que les « vraies » fonctions demandent une attention particulière et un code plus lourd. Dans le cas du programme que j'ai réalisé, de simples étiquettes avec une sauvegarde du registre ra qui permet de se souvenir où en était le programme suffit.

Fonctions que j'ai implémenté :

- Initialiser grille :
    - Prend en paramètre l'adresse du registre où un espace de 16 octets a préalablement été réservé ;
    - Initialise un tableau de 16 entiers chacun à 0, ces tableaux seront les grilles du joueur et de l'ordinateur ;
    - Ne renvoie rien.

- Demander coordonnées :
    - Prend en paramètre le message à afficher ;
    - Demande une coordonnée compris en 0 et 3, si l'utilisateur entre un nombre qui n'est pas compris entre 0 et 3, celui est forcé de le faire ;
    - Renvoie la coordonnée obtenue.

- Calculer index :
    - Prend en paramètre : X et Y ;
    - Transforme les coordonnées X,Y en un index compris entre 0 et 15 (index = X * 4 + Y) ;
    - Renvoie l'index obtenue.

- Vérifier tir :
    - Prend en paramètre : le tableau et l'index du tableau ;
    - Vérifie si le tir touche un bateau, l'eau ou s'il y a déjà eu un tir sur cette case ;
    - Renvoie 1, si le tir touche un bateau, ou 0 sinon.

- Vérifier proximité :
    - Prend en paramètre : le tableau, X et Y ;
    - Regarde s'il y a un bateau adjacent, affiche un message dans le cas où il y en a un. Prise en compte des « bords » de la grille ;
    - Ne renvoie rien.

Analyse de fonction complexes

- 1ère fonction, Demander coordonnée :

```
#Fonction qui demande une coordonnée
demander_coordonnee:
        mv t1, a0

boucle_saisie:
        mv a0, t1
        li a7, 4
        ecall
        li a7, 5
        ecall
#Test :
#<0 ?
        blt a0, zero, coord_invalide
#>3 ?
        li t2, 3
        bgt a0, t2, coord_invalide
        jalr zero, 0(ra)

#gestion erreur
coord_invalide:
        la a0, msg_erreur
        li a7, 4
        ecall
        j boucle_saisie
```

1 – Fonction Demander coordonnée

Comme dit précédemment, le but de cette fonction est de demander une coordonnée c'est-à-dire un entier entre 0 et 3.

Cette fonction ne prend qu'un paramètre, l'adresse mémoire du message à afficher en a0 . Et retourne l'entier donné par l'utilisateur dans a0.

La fonction fonctionne en plusieurs phase, premièrement la sauvegarde du message en t1, et l'affichage de ce message. Puis la saisi de l'utilisateur de l'entier. Enfin la vérification que l'entier soit compris entre 0 (avec « blt a0, zero, coord_invalide ») et 3 (avec « bgt a0, t2, coord_invalide »). Si les branchements sont pris, le programme est redirigé vers l'étiquette « coord_invalide », le programme affiche alors un message d'erreur et retourne au niveau de l'étiquette « boucle saisie » et recommence le processus de demande de coordonnée. Quand la coordonnée est correctement entrée par l'utilisateur on retourne dans le programme principalement avec la coordonnée dans le registre a 0.

- 2ème fonction, vérification proximité :

```
verifier_proximite:
#init de la fonction
        mv t0, a0
        mv t1, a1
        mv t2, a2
        li t6, 1

#verif gauche
        beq zero, t1, prox_droite    # Si X == 0, on ne peut pas aller à gauche
        addi t3, t1, -1
        slli t4, t3, 2
        add t4, t4, t2
        add t4, t0, t4
        lb t5, 0(t4)
        beq t5, t6, prox_trouve # Si 1 on passe directement au message


#verif droite
prox_droite:
        li t3, 3
        beq t1, t3, prox_haut # Si X == 3, on ne peut pas aller à droite
        addi t3, t1, 1
        slli t4, t3, 2
        add t4, t4, t2
        add t4, t0, t4
        lb t5, 0(t4)
        beq t5, t6, prox_trouve # Si 1 on passe directement au message

#verif haut
prox_haut:
        beq zero, t2, prox_bas # Si Y == 0, on ne peut pas aller en haut
        addi t3, t2, -1
        slli t4, t1, 2
        add t4, t4, t3
        add t4, t0, t4
        lb t5, 0(t4)
        beq t5, t6, prox_trouve # Si 1 on passe directement au message

#verif bas
prox_bas:
        li t3, 3
        beq t2, t3, prox_fin    # Si Y == 3, pas de bas
        addi t3, t2, 1
        slli t4, t1, 2
        add t4, t4, t3
        add t4, t0, t4
        lb t5, 0(t4)
        beq t5, t6, prox_trouve # Si 1 on passe directement au message

        j prox_fin              # Si on arrive ici, il n'y a pas de bateau adjacent

prox_trouve: #affichage qu'il y a un bateau
        la a0, msg_proche
        li a7, 4
        ecall

prox_fin:
        jalr zero, 0(ra)
```

2 - Fonction Vérifier proximité

Le but de cette fonction est d'indiquer si il y a des bateaux sur les cases adjacentes à la case ciblé dans le mode débutant.

Cette fonction prend trois paramètres : l'adresse mémoire de la grille dans a0, la coordonnée X dans a1, et la coordonnée Y dans a2. Elle ne retourne rien, elle ne fait qu'afficher un message dans le cas où il y a un bateau.

La fonction fonctionne en plusieurs étapes, d'abord une initialisation puis un test pour les quatre directions. Pour chaque direction, par exemple la gauche, elle vérifie d'abord si la case n'est pas située sur un bord de la grille (avec « beq zero, t1, prox_droite ») pour éviter de lire en dehors du tableau. Si la case ne prend pas le branchement, elle calcule l'index de la case adjacente, lit sa valeur en mémoire (avec « lb t5, 0(t4) »), et vérifie si elle contient un bateau

(c’est-à-dire si la valeur est 1). Si c'est le cas (avec « beq t5, t6, prox_trouve »), le programme prend un branchement vers l'étiquette « prox_trouve » qui affiche le message.

Si la case adjacente ne contient pas de bateau (valeur 0), le programme passe simplement à la vérification de la direction suivante (droite, puis haut, puis bas). Si aucun bateau n'est trouvé autour après les quatre tests, le programme saute à l'étiquette « prox_fin » et retourne dans le programme principal.

Pour conclure j’ai rencontré deux grandes difficultés.

Tout d’abord, le manque de ressource complémentaire au cours sur internet, notamment pour générer un entier aléatoire j’ai épluché plusieurs forums *Redit* avant de trouver un site (https://asm-editor.specy.app/documentation/risc-v/syscall)qui explique qu’il faut utiliser la fonction ecall avec le nombre 42 dans a7 et les borne inférieur et supérieur dans a0 et a1.

Le deuxième point qui m’a causé problème a été de savoir dans quels registres était mes variables, ce qui était ma principale cause de bug lors de mes tests. Pour remédier à cela j’ai écrit à la main sur une feuille à part ce que contenait chacun de des registres utilisés.